

**ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА**

**ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №1
«Організація професійного Python-проєкту та Git»
з курсу «Професійний Python»
Варіант №5 — «Система обліку студентів»**

Виконала: Хілінська Іванна

Група: ФєП-32

Перевірів: Куньо Іван Михайлович

Дата: 21.09.2026

Львів — 2026

1. Мета роботи

Ознайомитися з основними принципами організації професійного Python-проєкту, навчитися створювати ізольоване середовище розробки, використовувати src-layout, модулі та пакети Python, файл pyproject.toml, Git і базову документацію. Практично реалізувати консольний застосунок за індивідуальним варіантом №1.

2. Індивідуальний варіант і постановка задачі

Варіант №5 — «Каталог автомобілів». Для кожного автомобіля зберігати марку (make), модель (model), рік випуску (year), ціну (price) та пробіг (mileage). Реалізувати додавання автомобіля, виведення списку автомобілів, пошук за маркою, фільтрацію за роком випуску, визначення найдорожчого автомобіля, обчислення середньої ціни та пошук авто з найменшим пробігом.

Проєкт повинен мати src-layout, щонайменше три Python-модулі (models.py, services.py, main.py), дата-модель (data model), окремий модуль із бізнес-логікою (business logic), а також type hints, README.md, .gitignore, pyproject.toml та Git-репозиторій.

3. Структура проєкту

```
car_catalog/  
├─ pyproject.toml  
├─ README.md  
├─ .gitignore  
├─ report/  
│   └─ Прізвище_Ім'я_Лаб01_V5.pdf  
├─ tests/  
│   └─ test_services.py  
└─ src/  
    └─ car_catalog/  
        ├── __init__.py  
        ├── main.py  
        ├── models.py  
        └─ services.py
```

Каталог src/car_catalog містить пакет застосунку. Файл models.py описує модель даних Car, services.py містить бізнес-логіку керування каталогом, main.py є точкою входу в консольний застосунок, __init__.py ініціалізує пакет, а tests/test_services.py містить автоматизовані модульні тести для перевірки функцій бізнес-логіки.

4. Опис створених модулів

4.1. models.py

Для представлення автомобіля використано dataclass Car з полями make, model, year, price та mileage. Метод `__post_init__` перевіряє коректність даних, а властивість `full_title` формує повну назву автомобіля.

```
@dataclass(slots=True)

class Car:

    make: str

    model: str

    year: int

    price: float

    mileage: int

    def __post_init__(self) -> None:

        if self.year < 1886:

            raise ValueError("Release year cannot be earlier than 1886.")

        if self.price < 0:

            raise ValueError("Price cannot be negative.")

        if self.mileage < 0:

            raise ValueError("Mileage cannot be negative.")

    @property

    def full_title(self) -> str:

        return f"{self.make} {self.model} ({self.year})"
```

4.2. services.py

У модулі реалізовано функції `add_car()`, `search_by_make()`, `filter_by_year()`, `find_most_expensive_car()`, `calculate_average_price()` та `find_lowest_mileage_car()`. Business logic не залежить від консольного інтерфейсу.

```
def add_car(cars: list[Car], car: Car) -> None:

    cars.append(car)

def search_by_make(cars: list[Car], make: str) -> list[Car]:
```

```

    return [c for c in cars if c.make.lower() == make.lower()]

def filter_by_year(cars: list[Car], min_year: int) -> list[Car]:
    return [c for c in cars if c.year >= min_year]

def find_most_expensive_car(cars: list[Car]) -> Car | None:
    return max(cars, key=lambda c: c.price, default=None)

def calculate_average_price(cars: list[Car]) -> float:
    if not cars:
        return 0.0
    return sum(c.price for c in cars) / len(cars)

def find_lowest_mileage_car(cars: list[Car]) -> Car | None:
    return min(cars, key=lambda c: c.mileage, default=None)

```

4.3. main.py

Модуль main.py створює демонстраційні дані, викликає функції business logic та виводить результати. Конструкція if __name__ == "__main__": відокремлює точку входу від функцій програми.

```

from car_catalog.models import Car

from car_catalog.services import (
    add_car,
    search_by_make,
    filter_by_year,
    find_most_expensive_car,
    calculate_average_price,
    find_lowest_mileage_car,
)

def main() -> None:
    cars: list[Car] = [
        Car("Toyota", "Camry", 2019, 21000.0, 75000),
        Car("BMW", "X5", 2021, 55000.0, 30000),
    ]

```

```

        Car("Toyota", "RAV4", 2022, 32000.0, 15000),

        Car("Audi", "A6", 2018, 23000.0, 110000),

    ]

    print("--- All Cars in Catalog ---")

    for car in cars:

        print(f"{car.full_title} - ${car.price:.2f}, mileage: {car.mileage} km")

    avg_price = calculate_average_price(cars)

    print(f"\nAverage car price: ${avg_price:.2f}")

    most_expensive = find_most_expensive_car(cars)

    if most_expensive:

        print(f"Most expensive car: {most_expensive.full_title} (${most_expensive.price:.2f})")

    lowest_mileage = find_lowest_mileage_car(cars)

    if lowest_mileage:

        print(f"Lowest mileage car: {lowest_mileage.full_title} ({lowest_mileage.mileage} km)")

if __name__ == "__main__":

    main()

```

4.4. test_services.py

Додано pytest-тести для пошуку за маркою, фільтрації за роком випуску, обчислення середньої ціни, визначення найдорожчого авто, авто з найменшим пробігом та валідації некоректних даних.

5. Файл pyproject.toml

Файл задає build system setuptools, метадані проєкту, Python 3.11+, тестову залежність pytest та параметри конфігурації pytest.

```

[build-system]

requires = ["setuptools>=70"]

build-backend = "setuptools.build_meta"

```

```
[project]

name = "car-catalog"

version = "0.1.0"

description = "Car catalog laboratory project"

requires-python = ">=3.11"

optional-dependencies = { test = ["pytest>=8.0"] }


[project.scripts]

car-catalog = "car_catalog.main:main"


[tool.pytest.ini_options]

pythonpath = ["src"]

testpaths = ["tests"]
```

6. Файл .gitignore

З Git виключено virtual environment, кеші, IDE configuration, build artifacts і .env.

```
.venv/
__pycache__/
*.pyc
.pytest_cache/
.idea/
.vscode/
dist/
build/
*.egg-info/
.env
```

7. Фрагмент README.md

Car Catalog

Лабораторна робота №1 з курсу «Професійний Python», варіант №5.

Installation

```
python -m venv .venv  
python -m pip install -e ".[test]"
```

Run

```
python -m car_catalog.main
```

Tests

```
python -m pytest
```

8. Створення virtual environment та запуск

```
python -m venv .venv  
.venv\Scripts\activate  
python -m pip install -e ".[test]"  
python -m car_catalog.main
```

9. Результат роботи програми

Фактичний результат запуску згенерованого проєкту:

```
--- All Cars in Catalog ---  
Toyota Camry (2019) - $21000.00, mileage: 75000 km  
BMW X5 (2021) - $55000.00, mileage: 30000 km  
Toyota RAV4 (2022) - $32000.00, mileage: 15000 km  
Audi A6 (2018) - $23000.00, mileage: 110000 km  
  
Average car price: $32750.00  
Most expensive car: BMW X5 (2021) ($55000.00)  
Lowest mileage car: Toyota RAV4 (2022) (15000 km)
```

10. Робота з Git

У демонстраційному проєкті створено три логічні commits.

```
git init  
git add .gitignore pyproject.toml src/car_catalog/__init__.py  
git commit -m "Create project structure"
```

```
git add src/car_catalog/models.py src/car_catalog/services.py tests/test_services.py

git commit -m "Add car model and business logic"
```

```
git add src/car_catalog/main.py README.md

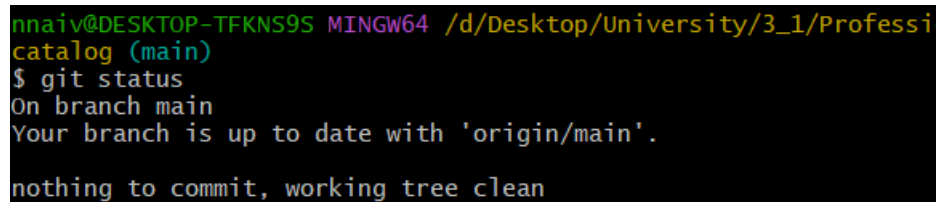
git commit -m "Add application entry point and documentation"
```

10.1. Результат git status

On branch main

Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean



```
nnaiv@DESKTOP-TFKNS9S MINGW64 /d/Desktop/University/3_1/Professi
catalog (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

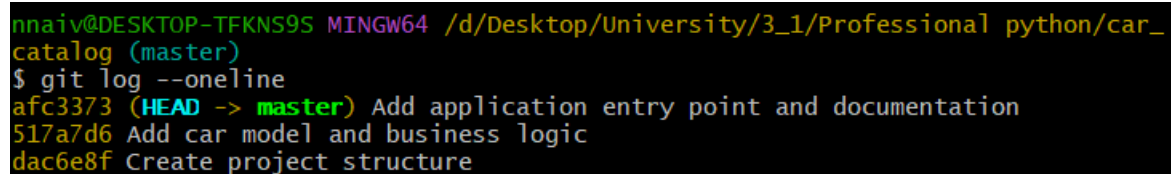
nothing to commit, working tree clean
```

10.2. Результат git log --oneline

afc3373 (HEAD -> master) Add application entry point and documentation

517a7d6 Add car model and business logic

dac6e8f Create project structure



```
nnaiv@DESKTOP-TFKNS9S MINGW64 /d/Desktop/University/3_1/Professional python/car_
catalog (master)
$ git log --oneline
afc3373 (HEAD -> master) Add application entry point and documentation
517a7d6 Add car model and business logic
dac6e8f Create project structure
```

11. Висновок

У лабораторній роботі створено структурований Python-проект за src-layout для каталогу автомобілів. Реалізовано data model Car, окремий модуль business logic, точку входу main.py, type hints, валідацію, фільтрацію, обчислення агрегованих показників та автоматизовані тести. Підготовлено pyproject.toml, .gitignore і README.md, а також сформовано логічну Git-історію з трьох commits. Отримано практичні навички організації професійного Python-проекту, роботи з virtual environment і Git.